

Clase 5_1:

Programación

Programación Orientada a Objetos

Temari

-

- • Composición de objetos

- Jerarquía de clases.

- Herencia

-

-

Composición de objetos

La composición es un tipo de **relación en la cual se expresa una dependencia entre dos objetos**, en donde **un objeto más complejo es conformado por objetos más pequeños**. En esta situación, se aplica la **relación “tiene un”**:

La computadora tiene un monitor.

El auto tiene ruedas.

El curso tiene alumnos

La forma usual de componer objetos a partir de otros más simples **es almacenarlos en atributos**.

Componer es una técnica fundamental de la programación orientada a objetos.

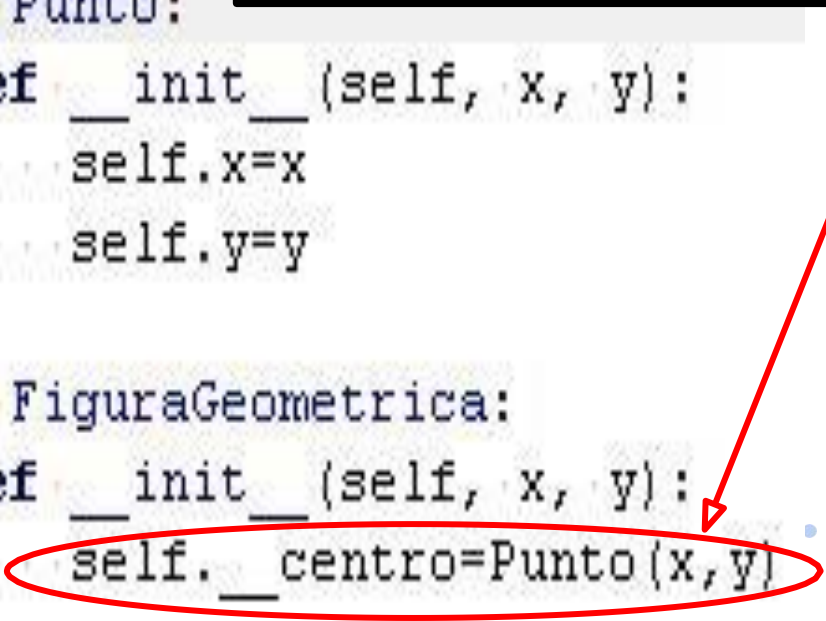
Composición de objetos

En el problema de dibujar figuras, la siguiente frase tiene sentido: “Las figuras geométricas tienen un punto que guarda el centro de coordenadas. ”

Las **instancias de FiguraGeometrica** están **compuestas** por **una instancia de la clase Punto** que guarda el centro de la misma.

```
class Punto:
    def __init__(self, x, y):
        self.x=x
        self.y=y
```

```
class FiguraGeometrica:
    def __init__(self, x, y):
        self._centro=Punto(x,y)
```



Composición de objetos

- Cuando se programa con objetos es usual crear **muchas clases de objetos**, donde **cada una tiene su estado y comportamiento** específico.
- Muchas veces **cuando un objeto es "muy grande"**, es decir tiene muchos estados y comportamientos, **es preferible crear varios objetos más chicos** y luego **un objeto que los englobe** a todos juntos.
- La composición de objetos implica que **un objeto está formado por otros**.

Composición de objetos

Supongamos que queremos definir una clase llamada **Dueño** que tenga un atributo privado llamado **nombre**, otro **apellido** y otro capaz de almacenar una **mascota**.
¿Cómo sería la implementación de la clase?

Mostraría composición de clases!

Composición de objetos

class Dueño:

```
def __init__(self, nom, ape,  
animal):
```

```
    self.__nombre = nom
```

```
    self.__
```

```
    __apellido = ape
```

```
    self.__suMascota=animal;
```

```
def verNom(self):
```

```
    return self.
```

```
    __nombre
```

```
def verApe(self):
```

```
    return self.
```

```
    __apellido
```

```
def verMascota(self):
```

```
    return self.
```

```
    __suMascota
```



Composición de objetos. En el estado de dueño interviene un objeto animal de la clase Mascota.

Composición de objetos

```
def modNom(self,  
    nuevo): self.  
    __nombre=nuevo
```

```
def modApe(self, otro):  
    self.  
    __apellido=otro
```

```
def modMascota(self,  
    nuevaMascota): self.  
    __suMascota=nuevaMascota
```

Composición de objetos

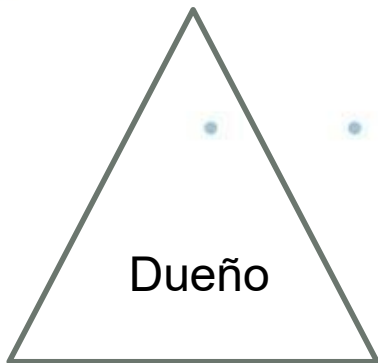
¿Cómo lo usamos en un programa de aplicación, que instancie un dueño, una mascota y los relacione (le asigne la mascota a su dueño)?.

```
mas = Mascota("Fido","perro","Juan",2);  
persona=Dueño("Juan", "Diaz", mas);
```

Esta sentencia genera la composición de objetos. La instancia dueño "conoce" a la instancia mascota.

Composición de objetos

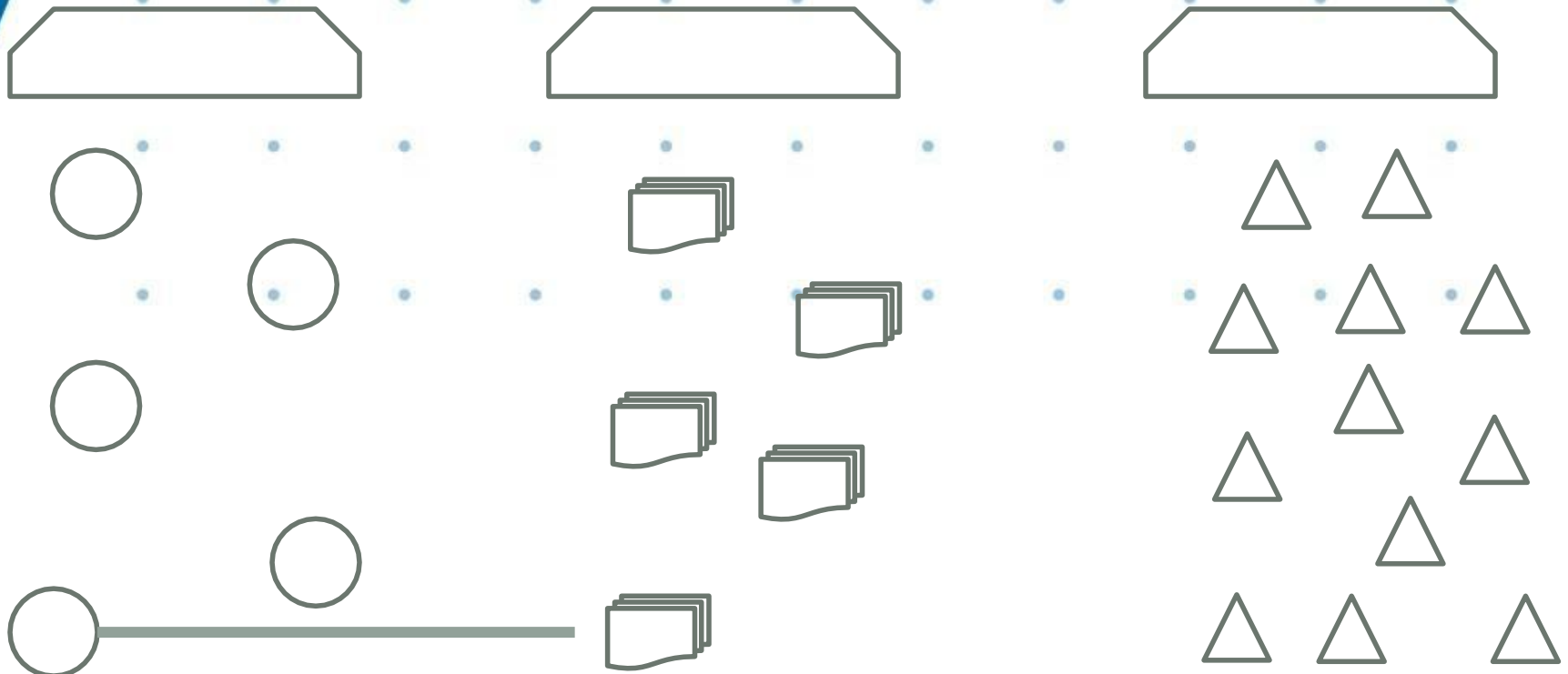
```
mas = Mascota("Fido", "perro", "Juan", 2);  
persona = Dueño("Juan", "Diaz", mas);
```



El dueño **"conoce"** a su mascota ¿La mascota "conoce" a su dueño? **No.**

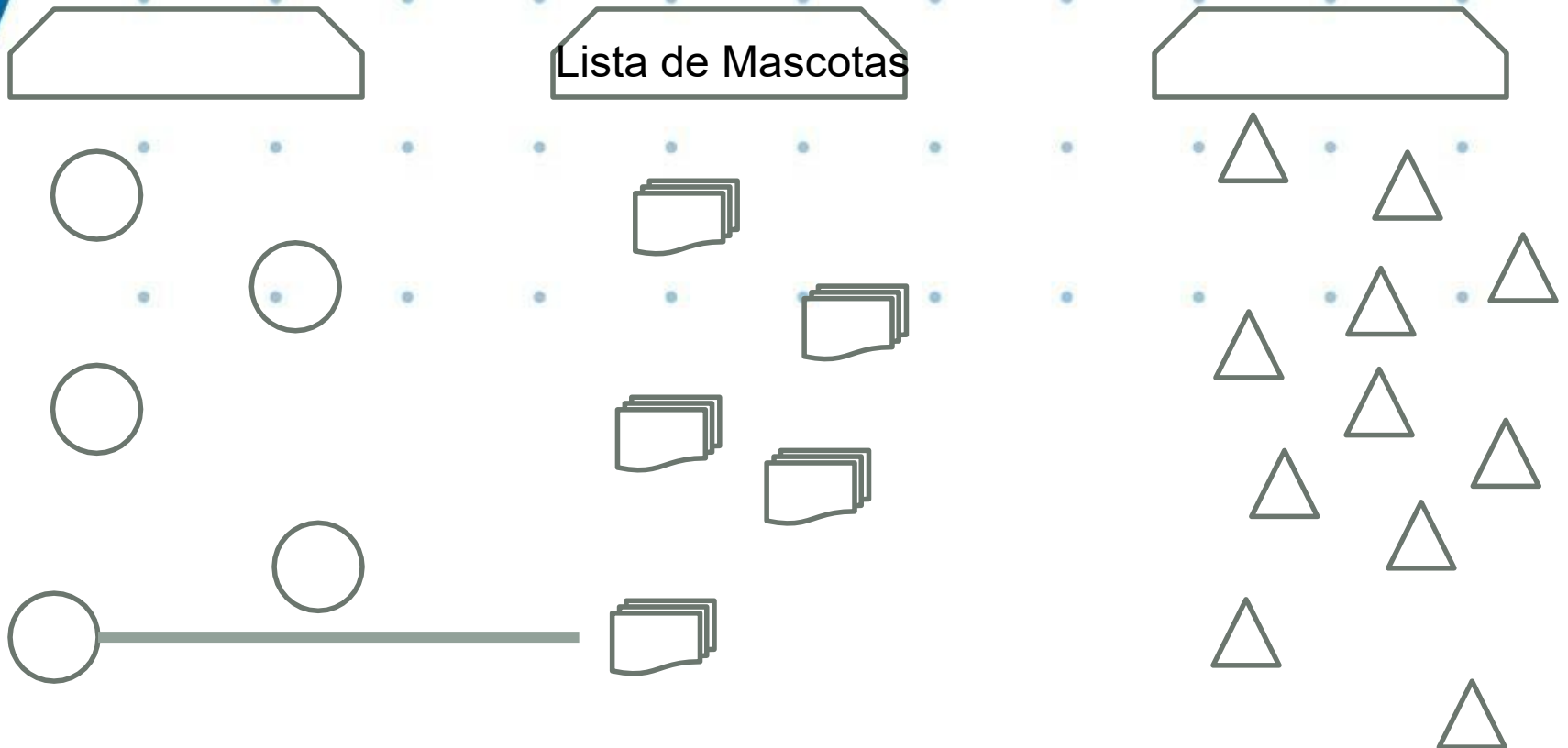
Composición de objetos

- Cada veterinaria (instancia) tiene su propia lista de mascotas



Composición de objetos

- Cada lista de mascotas puede tener 0 o más mascotas

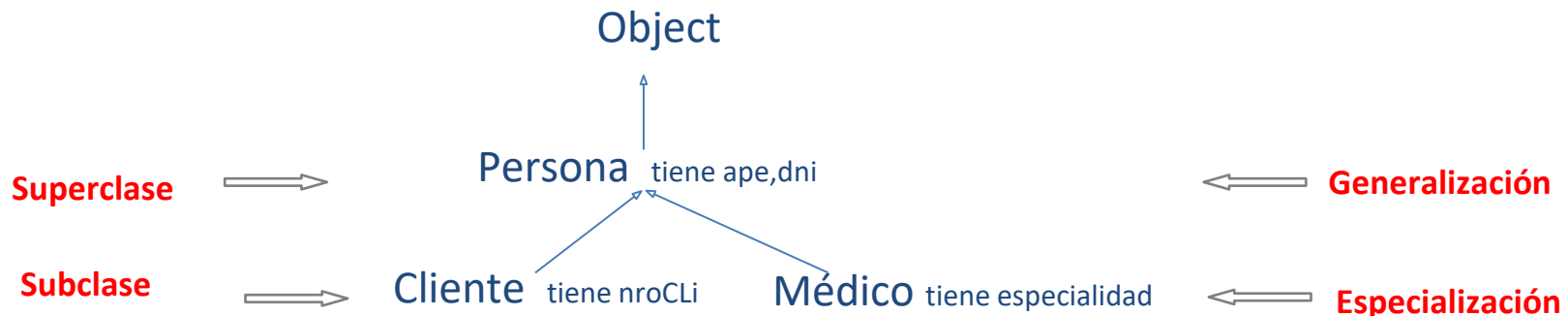


Jerarquía de clases y herencia.

La herencia nos permite concebir una nueva clase (subclase) de objetos como un refinamiento de otra clase de objetos ya existente (superclase), conservando las similitudes entre las clases de objetos y especificando solamente las diferencias de la nueva clase de objetos. Se arma un árbol jerárquico de clases.

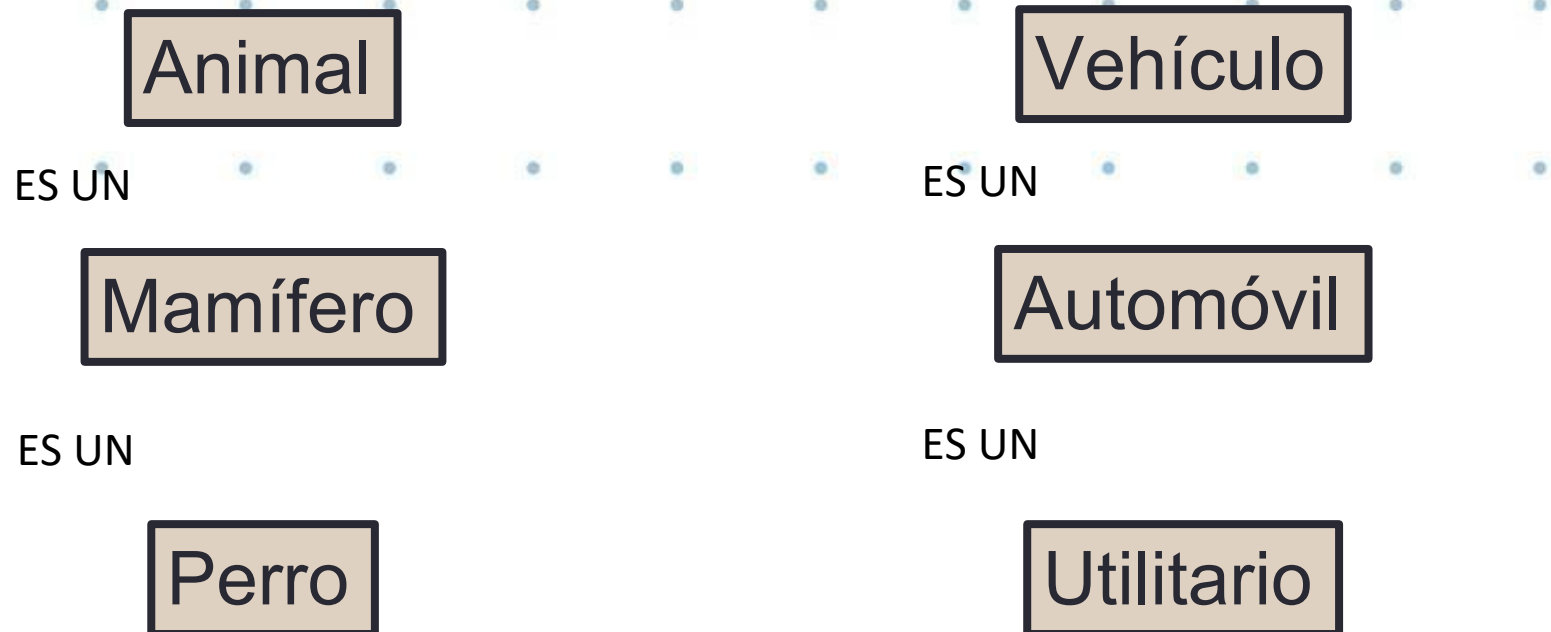
Las subclases pueden agregar nuevos atributos y comportamientos y además pueden cambiar el comportamiento de algunos métodos heredados.

Jerarquía de clases



Herencia

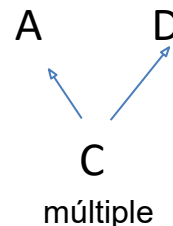
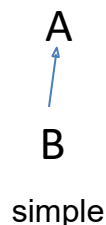
La herencia implementa la relación “*es un*” entre clases: nos permite establecer que una clase es como otra, con la **diferencia** de que la nueva clase incluye estado y comportamiento extra.



Herencia.

La herencia puede ser:

- **De estado:** se heredan los atributos de la superclase.
- **De comportamiento:** se heredan los métodos de la superclase.
- **Total o parcial:** se heredan todos los atributos y métodos(total), o solo parte de ellos.
- **Simple o múltiple:** se hereda de una sola superclase, o se hereda de varias superclases.



Herencia.

Cuando se programa usando herencia, se programa **por extensión:**

- Se extiende el código existente, se reutiliza Código. No se vuelve a escribir lo heredado. Sólo se agrega código *Correspondiente a los nuevos atributos y las nuevas operaciones.*
- Cuando se le envía un mensaje a un objeto, el S.O. lo busca en la clase de dicho objeto. Si no lo encuentra, porque es heredado, sube por el árbol jerárquico de clases aplicando una **BÚSQUEDA LOOK-UP**, hasta encontrarlo y lo ejecuta.

Herencia en Python

- Supongamos tener la clase Vehículo que representa la funcionalidad de un auto cualquiera.

```
class Vehiculo:
```

```
    def __init__(self, mar, mod):
```

```
        self.____marca=mar
```

```
        self.____modelo=mod
```

```
        self.____motor='apagado'
```

```
    def encenderMotor(self):pass
```

```
    def apagarMotor(self):pass
```

```
    def acelerar(self,vel): pass
```

```
    def frenar(self): pass
```

```
    def doblar(self,ang): pass
```

```
    def encenderLuces(self,cuales): pass
```

```
.....
```

Herencia en Python

- Si necesitamos **modelar un auto de policía**, que además de hacer todo lo que hace cualquier auto, **enciende la sirena**. ¿Qué **cambios** habría que hacer en la clase **Vehiculo**?
• • • • • • • • • •
- Si necesitamos **modelar un vehículo utilitario** que además de hacer todo lo que hace cualquier auto, **carga y descarga mercadería**. ¿Qué **cambios** habría que hacer en la clase **Vehiculo**?

Herencia en Python

- La clase **Vehiculo** tiene mucha **funcionalidad** implementada.
- Sería ideal poder usar esa **funcionalidad** sin hacer **modificación** alguna.
- Utilizar la clase **Vehiculo** para agregar **nueva funcionalidad** es lo que nos permite la **herencia** de clases.

Herencia en Python

Se crean las clases **MovilPolicial** y **Utilitario** como subclases de **Vehiculo**:

```
class MovilPolicial (Vehiculo):
    #constructor de MovilPolicial
    def encenderSirena(self): pass
    def apagarSirena(self): pass
    def verNroMov(self): pass
    def modNroMov(self,otroN): pass
    .....
class Utilitario (Vehiculo):
    #constructor de Utilitario
    def cargar(self,kg): pass
    def descargar(self,kg): pass
    def verCapacidad(self):pass
    def modCapacidad(self,otra): pass
```

Herencia en Python

- Se crean las clases MovilPolicial y Utilitario como subclases de Vehiculo.

```
class MovilPolicial ( Vehiculo):  
    #constructor de MovilPolicial  
    def encenderSirena(self): pass  
    def apagarSirena(self): pass
```

.....

```
class Utilitario (Vehiculo):  
    #constructor de Utilitario  
    def cargar(self,kg): pass  
    def descargar(self,kg): pass
```

.....

Las clases MovilPolicial y Utilitario son subclases de Vehiculo.

Vehiculo es la superclase de ambas.

Herencia en Python

- Se crean las clases MovilPolicial y Utilitario como subclases de Vehiculo.

```
class MovilPolicial ( Vehiculo):  
    #constructor de MovilPolicial  
    def encenderSirena(self): pass  
    def apagarSirena(self): pass  
    .....
```

```
class Utilitario (Vehiculo):  
    #constructor de Utilitario  
    def cargar(self,kg): pass  
    def descargar(self,kg):pass  
    .....
```

Las clases MovilPolicial y Utilitario heredan todo el estado y el comportamiento de la superclase Vehiculo

Herencia en Python

Las clases MovilPolicial y Utilitario son **utilizadas como cualquier otra clase.**

```
autito = Vehiculo("Kwid",2025)
```

```
mpoli = MovilPolicial("Corsa", 2012, 23)
```

```
util = Utilitario("Fiorino",2003, 45)
```

```
autito.encenderMotor()
```

```
mpoli.encenderLuces('posicion')
```

```
util.frenar()
```

Herencia en Python

Las clases `MovilPolicial` y `Utilitario` son utilizadas como cualquier otra clase.

```
autito = Vehiculo("Siena",2007)
```

```
mpoli = MovilPolicial("Corsa", 2012,23)
```

```
util = Utilitario("Fiorino",2003,45)
```

```
autito.encenderMotor()
```

```
mpoli.encenderLuces('posicion')
```

```
util.frenar()
```

Notar que la clase **MovilPolicial** no tiene un método llamado **encenderLuces**. Y la clase **Utilitario** no tiene un método llamado **frenar**



Herencia en Python

Las clases MovilPolicial y Utilitario son utilizadas como cualquier otra clase.

```
autito = Vehiculo("Siena",2007)
```

```
mpoli = MovilPolicial("Corsa", 2012,23)
```

```
util = Utilitario("Fiorino",2003,45)
```

```
autito.encenderMotor()
```

```
mpoli.encenderLuces('posicion')
```

```
util.frenar()
```

Tanto MovilPolicial como Utilitario heredan todos los métodos de la clase Vehiculo. Por lo tanto, a los objetos de estas dos clases se les puede enviar cualquier mensaje heredado de la clase Vehiculo

```
graph TD; V[Vehiculo] --> MP[MovilPolicial]; V --> U[Utilitario];
```

The diagram illustrates inheritance in Python. A box on the right contains text explaining that both `MovilPolicial` and `Utilitario` inherit all methods from the `Vehiculo` class. Two arrows point from this box to the method calls in the code: one points to `mpoli.encenderLuces('posicion')` and the other points to `util.frenar()`, indicating that these methods are inherited from `Vehiculo`.

Herencia en Python

Las clases MovilPolicial y Utilitario son utilizadas como cualquier otra clase.

```
autito = Vehiculo("Siena",2007)
```

```
mpoli = MovilPolicial("Corsa", 2012,23)
```

```
util = Utilitario("Fiorino",2003,45)
```

```
autito.apagarMotor()
```

```
mpoli.encenderSirenas()
```

```
util.cargar(25)
```



En cada subclase hay métodos propios.

Herencia en Python

Las clases MovilPolicial y Utilitario son utilizadas como cualquier otra clase.

```
autito = Vehiculo("Siena",2007)
```

```
mpoli = MovilPolicial("Corsa", 2012,23)
```

```
util = Utilitario("Fiorino",2003,45)
```

Dan
error

autito. encenderSirenas()

mpoli. descargar(30)

util.apagarSirenas()

**autito no entiende encenderSirenas
mpoli no entiende descargar y
util tampoco entiende apagarSirenas**

Herencia en Python

- Las clases **MovilPolicial** y **Utilitario** deben definir sus **propios constructores** y **reutilizan lo definido en la superclase**.

```
class MovilPolicial (Vehiculo):
```

```
    def __init__(self, mar, mod, nromov): super().__init__(mar, mod)
```

```
        self.__móvil=nromov
```

```
        self.__sirena='apagada'
```

```
        .....
```

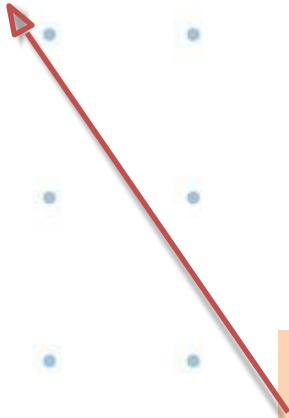
```
class Utilitario (Vehiculo):
```

```
    def __init__(self, mar, mod, maxcarga):
```

```
        super().__init__(mar, mod)
```

```
        self.__capac=maxcarga
```

```
        self.__cargaActual= 0
```



En cada constructor se invoca al constructor de la super clase primero.

Reutilizando código- Programación por

- Una **forma usual de reutilizar código**, es **invocar métodos con funcionalidad común** codificados en las superclases.

```
class Terrestre:
    def desplazar(self):
        print ("camina")

class Mamifero(Terrestre):
    def desplazar(self):
        super().desplazar()
        print ("el mamifero")
```

Invocación

Primero invoca el método de la superclase. Luego ejecuta el código de la propia subclase

```
m = Mamifero()
m.desplazar()
```

¿Cuál es el resultado?
Camina el mamífero

Ejercicio 1

Dada la clase Mascota, arme una jerarquía de clases donde las subclases sean las diferentes especies de animales.

-¿Cuál es la superclase? ¿Cómo se implementa?

Ejercicio 1

**Notar que
desaparece el
atributo especie**

```
class Mascota:
```

```
    def _____init_____(self, nom, ed, nomdue):
```

```
        self._____nombre=nom
```

```
        self._____edad=ed
```

```
        self._____duenio=nomdue
```

```
def verNom(self):  
    return self.  
        __nombre
```

```
def verEdad(self):  
    return self.  
        __edad
```

```
def verDuenio(self):  
    return self.  
        __duenio
```

```
def modNom(self, otroN):  
    self.  
        __nombre=otroN
```

```
def modDue(self, otroD):  
    self.
```

Ejercicio 2

¿Cómo se implementan las subclases?

Ejercicio 2

```
class Perro ( Mascota)
```

```
def __init__(self, nom, ed, due, raza): super().
```

```
    __init__(nom,ed, due)
```

```
    self.__raza=raza
```

```
def verRaza(self): return self.__raza
```

```
def modRaza(self, otra):
```

```
    self.__raza=otra
```

```
class Gato (Mascota):
```

```
def __init__(self, nom, ed, due,colorpelo):
```

```
    super().__init__( nom, ed, due)
```

```
    self.__pelaje=colorpelo
```

```
def verPelo(self):
```

```
    return self.__pelaje
```

```
def modPelo(self,otroP):
```

```
    self.__pelaje=otroP
```

Cada clase define su propio estado, comportamiento y constructores nuevos

